

Document number	P3056R0
Date	2023-11-20
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Evolution Working Group (LEWG)

what ostream exception

Table of contents

- what ostream exception
 - Abstract
 - Motivational Example
 - Summary

Abstract

Currently there are a couple of implications of superfluous dynamic allocations associated with C++ exception handling when dealing with informative and dynamic error messages. This paper proposes that we allow programmers the ability to avoid these costs wherever possible.

Motivational Example

Following is a common example of the superfluous dynamic allocations presented in this paper.

```

#include <string>
#include <vector>
#include <iostream>
#include <source_location>
#include <sstream>

using namespace std::literals::string_literals;

template<class T>
class my_vector : public std::vector<T> {
private:
    std::string to_string(const std::vector<T>::size_type pos, const std::source_location location) const {
        std::stringstream ss;
        ss << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") ` "
            << location.function_name() << "`: "
            << "invalid index: " << pos << ", size: " << this->size() << '\n';
        return ss.str();
    }

    [[nodiscard]]
    constexpr bool test( std::vector<T>::size_type pos ) const {
        return pos < this->size();
    }
public:
    using std::vector<T>::vector;
    typedef std::vector<T>::value_type value_type;
    typedef std::vector<T>::size_type size_type;
    typedef std::vector<T>::reference reference;
    typedef std::vector<T>::const_reference const_reference;
#if __STDC_HOSTED__ == 1
    [[nodiscard]]
    constexpr reference at( size_type pos, const std::source_location location = source_location::current() ) const {
        if(test(pos))
        {
            return (*this)[pos];
        }
        else
        {
            std::string temp = to_string(pos, location); // POTENTIALLY UNNECESSARY
            throw std::out_of_range(temp); // POTENTIALLY COPYING OF std::string
        }
    }
#endif
};

```

```

    }
}
[[nodiscard]]
constexpr const_reference at( size_type pos, const std::source_location locati
{
    if(test(pos))
    {
        return (*this)[pos];
    }
    else
    {
        std::string temp = to_string(pos, location);// POTENTIALLY UNNECESSARY
        throw std::out_of_range(temp);// POTENTIALLY COPYING OF std::string
    }
}
#endif
};

int main()
{
    try
    {
        my_vector<int> myints{1};
        int& pv = myints.at( 1 );//throws
    }
    catch(const std::out_of_range& oor)
    {
        std::clog << oor.what() << '\n';
    }
    return 0;
}

```

In the `my_vector` library author's code are two potentially superfluous dynamic allocations. These allocations are hidden from the library user who writes the `main` function. These two allocations are a consequence of the `std::logic_error` and `std::runtime_error` along with their derived classes copying a string. This also requires that sourced string be dynamically allocated as a `std::string` when the error message is not static.

```

virtual const char* exception::what() const noexcept;

logic_error( const std::string& what_arg );
invalid_argument( const std::string& what_arg );
domain_error( const std::string& what_arg );
length_error( const std::string& what_arg );
out_of_range( const std::string& what_arg );

runtime_error( const std::string& what_arg );
range_error( const std::string& what_arg );
overflow_error( const std::string& what_arg );
underflow_error( const std::string& what_arg );
system_error( std::error_code ec, const std::string& what_arg );
format_error( const std::string& what_arg );

```

These two allocations could be mitigate by both the library author and library user with the addition of a few methods to the standard library.

```

virtual std::ostream& exception::what(std::ostream& os) const noexcept
{
    return os << std::exception::what();
}

logic_error( std::string&& what_arg );// sink the error message
invalid_argument( std::string&& what_arg );// sink the error message
domain_error( std::string&& what_arg );// sink the error message
length_error( std::string&& what_arg );// sink the error message
out_of_range( std::string&& what_arg );// sink the error message

runtime_error( std::string&& what_arg );// sink the error message
range_error( std::string&& what_arg );// sink the error message
overflow_error( std::string&& what_arg );// sink the error message
underflow_error( std::string&& what_arg );// sink the error message
system_error( std::error_code ec, std::string&& what_arg );// sink the error messa
format_error( std::string&& what_arg );// sink the error message

```

In the original example, the second superfluous allocation could be avoided with an explicit move.

```
// POTENTIALLY UNNECESSARY DYNAMIC ALLOCATION
std::string temp = to_string(pos, location);
// POTENTIALLY COPYING OF std::string
throw std::out_of_range(temp);
```

```
// POTENTIALLY UNNECESSARY D
std::string temp = to_string
throw std::out_of_range(std:
```

The first superfluous allocation can be delayed to the point of use which allows the end programmer to decide whether they want it streamed to a string, file or fixed length buffer.

```

#include <string>
#include <vector>
#include <iostream>
#include <source_location>
#include <sstream>

using namespace std::literals::string_literals;

class exception_v2 : virtual public std::exception {
public:
    virtual std::ostream& what(std::ostream& os) const noexcept
    {
        return os << std::exception::what();
    }
};

class logic_error_v2 : virtual public exception_v2 {
};

class out_of_range_v2 : virtual public logic_error_v2 {
};

template<class T>
class out_of_range_v2_1 : virtual public out_of_range_v2 {
private:
    const T pos;
    const T size;
    const std::source_location location;
public:
    out_of_range_v2_1(const T pos, const T size, const std::source_location locati
    {
    }
    virtual std::ostream& what(std::ostream& os) const noexcept override
    {
        return os << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") `"
            << location.function_name() << "`: "
            << "invalid index: " << pos << ", size: " << size << '\n';
    }
};

template<class T>
class my_vector : public std::vector<T> {
private:

```

```

[[nodiscard]]
constexpr bool test( std::vector<T>::size_type pos ) const
{
    return pos < this->size();
}
public:
    using std::vector<T>::vector;
    typedef std::vector<T>::value_type value_type;
    typedef std::vector<T>::size_type size_type;
    typedef std::vector<T>::reference reference;
    typedef std::vector<T>::const_reference const_reference;

#if __STDC_HOSTED__ == 1
[[nodiscard]]
constexpr reference at( size_type pos, const std::source_location location = s
{
    if(test(pos))
    {
        return (*this)[pos];
    }
    else
    {
        throw out_of_range_v2_1<size_type>(pos, this->size(), location);
    }
}
[[nodiscard]]
constexpr const_reference at( size_type pos, const std::source_location locati
{
    if(test(pos))
    {
        return (*this)[pos];
    }
    else
    {
        throw out_of_range_v2_1<size_type>(pos, location);
    }
}
#endif
};

int main()
{
    try
    {
        my_vector<int> myints{1};
        int& pv = myints.at( 1 );//throws
    }

```

```
catch(const out_of_range_v2_1<std::vector<int>::size_type>& oor)
{
    oor.what(std::clog) << '\n';
    // OR
    //char output[30]{};
    //ostream os{span<char>{output}};
    //oor.what(os) << '\n';
}
return 0;
}
```

Summary

With several extra overloads we can minimize dynamic allocations when performing exception handling. Instead of these superfluous allocations being hidden, they are placed in the control of exception throwers and exception consumers where it belongs.